Launch File

Thumrongsak Kosiyatrakul
tkosiyat@cs.pitt.edu

Multiple Processes for One Application

- Recall that you may have to open multiple console and execute multiple commands just to run one robot application
- Example: Project 2
  - Terminal 1: ROS Master

```
roscore
```

  - Terminal 2: Kobuki Node

```
roslaunch kobuki_node minimal.launch
```

  - Terminal 3: `gscam` Node

```
roslaunch gscam v4l.launch
```

  - Terminal 4: `cmvision` Node

```
roslaunch mypackage cmvision.launch
```

  - Terminal 5: Constant command

```
rosrun mypackage constant_command.py
```

  - Terminal 6: Control node for project 2

```
rosrun mypackage project2.py
```

Multiple Processes for One Application

- ROS allows you to execute one single command which results in executing multiple processes
- You need a **launch** file
- In ROS, a launch file is a text file with XML style (tags) with the extension `.launch`
- Traditionally located in the `launch` directory under your package

```
/home/student/cs1567/src/mypackage/launch
```

- But it can be located anywhere under your package (`mypackage`) directory

The <launch> Tag

- All content of the launch file must be surrounded by the <launch> tag:

```
<launch>
    <!-- This is a comment -->




</launch>
```

- A comment must be surrounded by <!-- and --> as shown above

The <include> Tag

- To let the launch file execute another launch file, use the <include> tag

```
<launch>
    :
    <!-- roslaunch kobuki_node minimal.launch -->
    <include file="$(find kobuki_node)/launch/minimal.launch"/>
    :
</launch>
```

- The $(find kobuki_node) will result in the path to the location of the kobuki_node package
- In case of the kobuki_node package, the launch file is located under the launch directory

The <include> Tag

- For other packages, simply navigate to the package's directory using `roscd` and look for it
- For example, `gscam`:

```
student@robot:/home/student$ roscd gscam
student@robot:/opt/ros/kinetic/share/gscam$ ls
cmake     gscam_nodeletlaunch      nodelet_plugins.xml  v4l.launch
examples  nodelet_pipeline.launch  package.xml
```

- Note that launch files of `gscam` package are not in the `launch` directory
- Thus, for the `gscam` node:

```
<launch>
    :
    <!-- roslaunch gscam v4l.launch -->
    <include file="$(find gscam)/v4l.launch"/>
    :
</launch>
```

The <node> Tag

- To let the launch file execute a node, use the <node> tag

```
<launch>
    :
    <!-- rosrun mypackage constant_command.py -->
    <node name="constant_command" pkg="mypackage"
          type="constant_command.py"/>
    :
</launch>
```

- The name indicates the name of the node
  - In most case, it should be the same as the in `rospy.init_node()`
  - Different name will overwrite the node name when it is initialized
- The pkg indicates the name of the package
- The type indicates the name of the executable file

The <node> Tag

- Let's put them all together for project 2 (p2.launch)

```
<launch>
    <!-- roslaunch kobuki_node minimal.launch -->
    <include file="$(find kobuki_node)/launch/minimal.launch"/>

    <!-- roslaunch gscam v4l.launch -->
    <include file="$(find gscam)/v4l.launch"/>

    <!-- roslaunch mypackage cmvision.launch -->
    <include file="$(find mypackage)/cmvision.launch"/>

    <!-- rosrun mypackage constant_command.py -->
    <node name="constant_command" pkg="mypackage" type="constant_command.py"/>

    <!-- rosrun mypackage project2.py -->
    <node name="project2" pkg="mypackage" type="project2.py" output="screen" cwd="node"/>
</launch>
```

- Notes:
  - If your node needs to show output on the console screen, you need to add output="screen" as shown above
  - The cwd="node" specifies the working directory to be the location of the executable file

The `launch` File

- To use the previous launch file:

```
roslaunch mypackage p2.launch
```

- You do not need to run the `roscore` command
  - If the `roscore` is not currently running, the launch file will also execute the `roscore` command
- You only need one console screen
- If all your nodes support `Ctrl-C`, by pressing `Ctrl-C`, it will kill all program executed by the launch file